

小知识：如何观察应用程序的系统调用

除了 `write` 以外，我们平时编写的程序实际上已经用到了许多其他的系统调用。我们可以通过 `strace` 工具来查看一个程序在运行过程中使用了哪些系统调用。这里我们依然以前文的 `hello` 程序为例来查看它使用过的系统调用（见输出记录 3.1）。可以看到，在 Linux 的 shell 中执行这段 `hello` 程序时，首先 `execve` 系统调用被用来从文件系统中加载 `hello` 程序。此后，这段程序通过 `write` 系统调用来向终端输出字符串并最终显示到屏幕上。在这里，1 表示标准输出，13 则表示一共写了 13 个字符。这段程序执行完毕后，通过 `exit_group` 系统调用来退出自己。

输出记录 3.1 `hello` 程序使用的系统调用

```
[user@osbook ~] $ strace ./hello
execve("./hello", ["./hello"], 0x7ffc2e7d60 /* 28 vars */) = 0
...
write(1, "Hello World!\n", 13Hello World!)      =
13
exit_group(0)                                   = ?
...
```

3.2.6 系统调用的优化

系统调用作为应用程序调用操作系统的入口，其性能也非常重要。然而，不同于传统的函数调用，系统调用的过程复用了异常机制，因此不可避免地需要执行特权级切换、上下文保存等操作，导致其时延比普通的函数调用高 1 到 2 个数量级。对于需要频繁进行系统调用的应用来说，这是很大的性能开销。

那么，怎么才能绕过费时的异常机制来实现系统调用呢？可以通过在用户态和内核态之间共享一小块内存的方式，在应用与内核之间创建一条新的通道。具体来说又可以分为两种方法。

第一种方法是内核将一部分数据通过只读的形式共享给应用，允许应用直接读取。例如，`gettimeofday` 是一个常用的系统调用，用于获取当前的时间，可精确到微秒。传统的实现是：内核内部维护一个变量，以微秒为单位记录当前时间，在每次处理硬件时钟中断的时候更新这个变量；当发生系统调用时，内核读取并返回这个变量的值。为了绕过异常机制，内核可以将这个变量放在一个特殊的内存页中，这个内存页对应用来说是可读但不可写的。在 `libc` 中，对 `gettimeofday` 的实现仅仅是读取这个变量，然后返回变量的值，而不需要陷入内核，从而大大降低了时延。对于应用程序来说，由于无法修改这个值，所以也不会影响系统整体的安全性。Linux 的 `vDSO`（`virtual Dynamic Shared Object`）机制就是基于这种方法。这种方法的缺点在于，如果系统调用需要修改内核中的变量，或者在运行过程中需要读取更多内核数据，该方法就不适用了。

第二种方法则是允许应用以“向某一块内存页写入请求”的方式发起系统调用，并通过轮询来等待系统调用完成。内核同样通过轮询来等待用户的请求，然后执行系统调用，并将系统调用的返回值写入同一块内存页以表示完成。但应用和内核怎么能同时轮询呢？这个设计的关键点在于：让内核独占一个 CPU 核心，这个核心一直在内核态运行，而其他 CPU 核心则一直在用户态运行。这样，从系统整体来看，对于任何一个 CPU 核心，都不会发生从用户态到内核态的切换。这种设计可以大大降低系统调用的时延：在应用将请求写入内存页后的下一个时钟周期，处于轮询状态的内核立即可以读到这个请求，并开始运行处理函数；同样，当内核将返回结果写入内存页后，在另一个 CPU 核心处于轮询状态的应用立即可以读到结果并继续运行。FlexSC^[2]使用了这种方法。

然而，第二种方法同样存在缺点。第一个缺点在于，如果某个应用发起系统调用请求的时候，内核正在处理上一个系统调用，则时延依然会很长；如果有多个应用同时发起请求，内核需要一个个顺序处理，则时延可能比原来更长。导致这个问题的原因在于内核没能充分使用多核。解决方法也很直接：让多个 CPU 核心同时运行在内核态并轮询用户的请求。核心的数量可以根据具体的负载确定：当内核忙不过来时，占用的核心多一些，反之则少一些。例如，假设主机一共有 16 个 CPU 核心，当 1 个核心运行内核态不够时，可以增加到 2 个或更多核心专门运行内核代码，剩下的 14 个则一直运行用户态代码。

第二个缺点在于，如果整个系统只有一个 CPU 核心，那么该怎么办？一种直观的方法是将轮询改成批处理。当 CPU 运行在用户态时，应用程序一次发起多个系统调用请求（这里可以基于用户态的调度实现用户线程的切换），同样将请求和参数写入共享内存页。然后 CPU 切换到内核态，内核一次性将所有系统调用处理完，把结果写入共享内存页，再切换回用户态运行。这种方法的好处并不在于时延（事实上对于每个系统调用来说，时延反而可能变长），而是在于系统的整体吞吐率：对于系统整体来说，由于特权级的切换次数变少了，所以在单位时间内的有效工作量增加了。

3.3 操作系统提供的基本抽象与接口

本节主要知识点

- 为什么需要进程这一抽象？
- 相比直接使用物理内存地址，使用虚拟内存地址有哪些优势？
- 为什么说“一切皆文件”？

基于异常机制和系统 ISA，操作系统为应用程序提供底层硬件的抽象与接口，使应

用程序能够更方便地使用硬件，包括处理器、物理内存和设备，如图 3.14 所示。其中，对处理器的抽象是进程。操作系统将多个应用程序通过分时复用的方式在单个处理器上交替执行，使这些应用看上去像是“同时”在运行。之后提出了更轻量级的线程抽象以及多线程机制，允许一个应用程序并行使用多个处理器。对物理内存的抽象是虚拟内存，也就是对物理内存的虚拟化。应用程序可以使用与操作物理内存相同的接口（如 `ldr` 和 `str` 指令）来操作虚拟内存，而不用考虑物理内存空间是否足够，与其他应用同时使用内存是否会产生冲突等问题。对存储、网络、键盘、显示器等外部设备的抽象是文件。文件有一组很直观的操作接口，包括打开、读取、写入、关闭等。将设备抽象为文件，可使应用程序通过同一套接口来操作不同的设备。

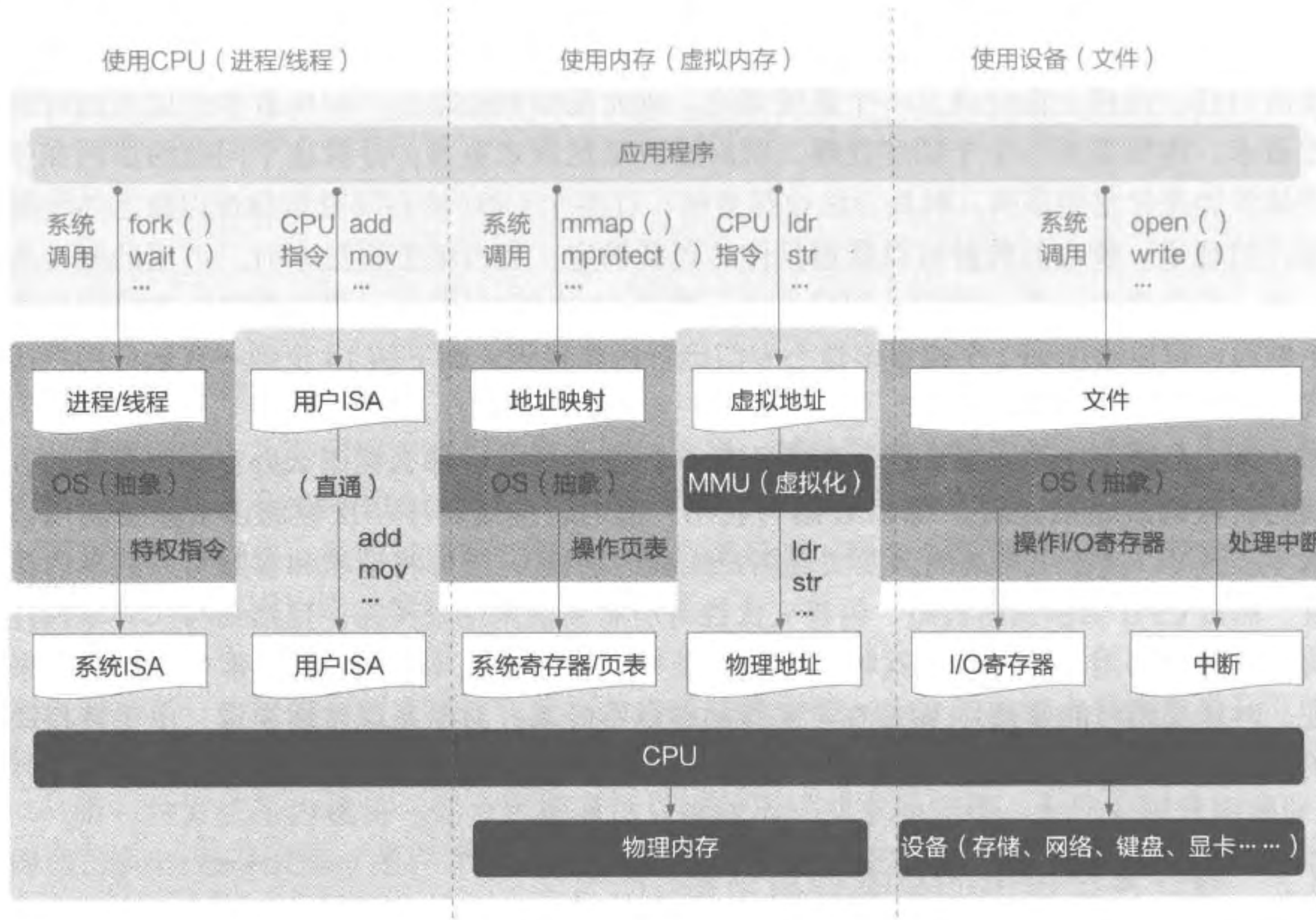


图 3.14 操作系统向应用提供的三种抽象

通过这些抽象和接口，应用程序可以使用操作系统提供的一系列功能。其中，有些功能对应用程序来说不是直接感知的，例如处理器的分时复用、虚拟地址映射等；有些功能则需要应用程序通过系统调用主动发起，例如创建进程、打开文件等。操作系统提供的这些功能，使程序员不用考虑硬件资源管理的繁杂细节，降低了应用程序的编程复杂度。本章将从程序员的视角来介绍这些抽象与接口。

抽象与虚拟化

抽象与虚拟化都是通过增加一个“层次”（即 indirection）来提供新的能力。通常来说，抽象的范围更广一些，而虚拟化可以被看作一种特殊的抽象：当抽象提供的接口与已存在的某一种接口相同时，这种抽象称为该接口的虚拟化。例如，虚拟内存机制提供的接口与物理内存一样，都是 `ldr` 和 `str` 指令，是对物理内存的虚拟化；虚拟机则是通过一层软硬件向上提供物理机的接口，是对物理机的虚拟化；如果有一层软件能够在 Linux 中提供 Windows 的系统调用接口，则是对 Windows 接口的虚拟化——比如 WINE^①。

3.3.1 进程：对处理器的抽象

现代操作系统中常常有数十上百个应用程序同时运行，然而处理器的个数是有限的。如果允许某几个程序持续“霸占”处理器，便会导致其他程序无法得到执行的机会。此外，运行中的应用程序并非每时每刻都需要使用处理器。例如，一个程序想要从磁盘上读取数据并进行处理，必须等到数据读取完成才能继续执行后续指令；由于磁盘的读取速度比处理器执行指令的速度慢很多，在程序等待磁盘数据的这段时间，处理器就被浪费了。那么，我们如何设计一种机制，让多个应用程序高效地、按需地、尽可能公平地运行在有限的处理器上呢？

操作系统的做法是让多个程序“轮流”使用处理器，又称为分时复用。具体来说，一个程序在处理器上运行一段时间后，会被操作系统暂停执行并切换走，把处理器让给另一个程序使用。切换的时机往往是在操作系统处理完某个异常或中断之后，且在返回用户态之前。操作系统设置每个程序每次最长连续运行的时间（称为时间片）通常很短，一般在毫秒级，这样就算有上百个活跃的程序，在 1 秒内也都有机会运行。通过这种高频切换运行的机制，既可以让多个活跃的程序从宏观来看是“同时”在运行，也可以使处理器尽可能保持满负载而不被浪费。同时，这种切换对程序来说是不可见的：对程序来说，独占一个处理器运行和通过分时复用处理器运行在逻辑上是等价的^②。因此，程序员写程序代码时，不用考虑运行时该如何与其他程序共享处理器，从而简化了程序的编写。

为了更好地管理多个运行中的程序，操作系统提供了进程的抽象。进程即运行中的程序，操作系统会为每个进程记录进程标识号（Process ID, PID）、运行状态以及所占用

① 参见 <https://www.winehq.org/>。

② 严格来说有两点不同：一是运行时间不同，如果进程记录每条指令执行的准确时刻，会发现在某两条相邻的指令之间，处理器好像停顿了一段时间——这意味着发生了一次切换；二是应用程序的某些 bug 可能在分时复用的情况下更容易被触发，比如竞争条件。

的计算资源，这些信息记录在一个称为进程控制块（Process Control Block, PCB）的数据结构中，操作系统为每个进程维护一个 PCB。从应用程序员的视角来看，进程抽象为程序提供了“独享”的处理器和内存等资源，从而大大简化了程序的编写；操作系统则以进程为单位来管理程序的运行状态，分配计算资源。图 3.15 展示了多个进程共享处理器的例子。

在图 3.15 中，操作系统让两个进程轮流在处理器上执行，但从其中任一进程的角度来看，就像是自己独占处理器一样。进程切换的过程由操作系统控制，对进程来说是不可感知的。

在切换进程时，操作系统会完成一系列操作，其中一个重要的步骤是将上一个进程在处理器中的状态保存到对应的 PCB 中，然后将下一个进程的状态从其 PCB 中读出并恢复到处理器对应的寄存器中。这些运行状态就是我们在 3.2.4 节提到过的处理器上下文（见代码片段 3.6）。此外，操作系统还需要更新两个进程对应 PCB 中的运行状态，并更换内核栈等（在关于进程的章节中会详细介绍）。对于一个进程来说，从上一次被暂停执行到下一次恢复执行前，处理器的状态就像没有被动过一样，从而实现了连续使用处理器的假象。

在提供进程这一抽象的同时，操作系统也提供了一系列接口，允许用户对进程进行操作，如创建进程、让进程执行指定的程序、退出进程等。操作系统为了更好地管理进程，同时提供进程之间的互操作、通信等功能，一般会根据进程的创建关系，通过层次化的方式将所有进程组织起来。通常，进程的创建者称为父进程，被创建的新进程则称为子进程。每个子进程都有对应的父进程，一直往上可以追溯到系统启动的第一个进程（如 Linux 中的 init）。

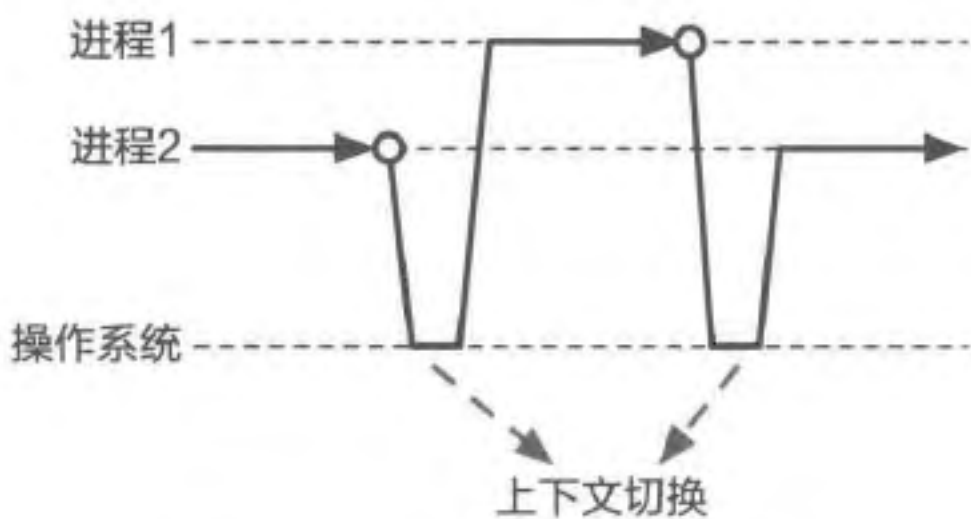


图 3.15 进程切换的例子

代码片段 3.6 操作系统在进程结构体中为每个进程维护上下文

```
1 struct process {
2     // 上下文
3     struct context *ctx;
4     ...
5 };
6 struct context {
7     // 处理器中的寄存器状态
8     unsigned long long reg[REG_NUM];
9     ...
10 };
```

3.3.2 案例分析：使用 POSIX 进程接口实现 shell

POSIX 规范提供了一组用于操作进程的接口，本节主要介绍其中四个常用的接口：spawn 接口用于创建新进程，waitpid 接口用于等待进程退出并回收资源，exit 接口

用于退出进程，getpid 接口用于获得当前进程的 ID。shell 便使用这些接口创建新的进程来执行用户命令，同时也能管理这些进程。

在本节中，我们将利用 POSIX 接口来实现一个只包含最基础功能的 shell：mysh。mysh 接收来自用户的命令，但一次只能执行一个程序，不支持参数传递，且无法强制终止程序的执行。mysh 的框架十分简单，如代码片段 3.7 所示，在启动后，mysh 会在循环内不断地打印命令行提示符 (mysh>) [⊖]，等待用户输入，并最终调用 eval 函数来执行用户输入的命令，直到其执行完成再进入下一轮循环。为了精简代码，本例未考虑错误处理。

代码片段 3.7 mysh 中的 main 函数

```

1 #include <stdio.h>
2 #include <spawn.h>
3 #include <sys/wait.h>
4 #include <string.h>
5 #include <stdlib.h>
6 #define MAXLINE 1000
7
8 int main()
9 {
10     char cmdline[MAXLINE];
11
12     while (1) {
13         printf("mysh> ");
14         fflush(stdout);
15
16         fgets(cmdline, MAXLINE, stdin);
17         //remove trailing newline character
18         cmdline[strcspn(cmdline, "\n")] = '\0';
19         eval(cmdline);
20     }
21     return 0;
22 }
23
24 void eval(char *cmdline)
25 {
26     char *argv[2] = {cmdline, NULL};
27     if (strcmp(cmdline, "quit") == 0) {
28         exit(0);
29     }
30
31     pid_t pid;
32     //create a new process to execute command
33     posix_spawn(&pid, cmdline, NULL, NULL, argv, NULL);
34     //wait for the new process to terminate
35     int exit_status;

```

⊖ 由于 printf 函数会将输出缓存起来，直到遇到换行符才真正进行输出，这里使用 fflush 函数强制进行输出。

```

36  waitpid(pid, &exit_status, 0);
37  if (!WIFEXITED(exit_status)) {
38      printf("Program terminated unexpectedly!\n");
39  }
40  }

```

mysh 的主体在于 eval 对用户输入的命令的处理。在真正执行用户命令之前，我们需要先判断用户的命令是否为 shell 的内置命令^①。mysh 只提供了一条内置命令 quit，用于退出 shell。一旦用户键入 quit，mysh 将调用 POSIX 中的 exit 接口来退出自己，也就是 shell 进程。与平时在 C 语言中使用的 return 不同，调用 exit 后进程将直接退出，不会再执行之后的代码。

对于非内置命令，mysh 将调用 POSIX 中的 posix_spawn 接口，创建一个新的进程来执行。在 posix_spawn 中，操作系统会自动根据参数中的可执行文件路径将其加载到内存，为其创建新进程并执行。

为了方便进程之间的操作，操作系统还为每个进程提供了唯一的 PID。PID 通常用来作为进程相关接口的参数，实现进程间的交互和通信。例如在 mysh 中，利用 posix_spawn 接口返回的子进程 PID，shell 进程便能通过 waitpid 接口等待子进程执行结束，并获得子进程的退出状态，从而在执行命令非正常退出时提醒用户。

来尝试运行一下 mysh 吧！hello-pid 是一个输出当前进程 PID 的简单程序。我们将通过 mysh 来执行三个命令：先通过 /usr/bin/ls 列出当前目录下的文件，然后执行 hello-pid 程序，最后通过内置命令 quit 退出 mysh。输出结果如输出记录 3.2 所示。

输出记录 3.2 一个简单的使用 mysh 的例子

```

1 //file: hello-pid.c
2 #include <stdio.h>
3 #include <unistd.h>
4 int main ()
5 {
6     printf("hello! my pid is %d\n", getpid());
7 }

```

```

[user@osbook ~/mysh] $ ./mysh
mysh> /usr/bin/ls
hello-pid  hello-pid.c  mysh  mysh.c

mysh> ./hello-pid
hello! my pid is 29774.

```

① shell 通常会内置一些命令，如 cd（进入某个目录，全称为 Change Directory）。


```
mysh> quit  
[user@osbook ~] $
```

3.3.3 虚拟内存：对内存的抽象

接下来，我们思考如何让多个进程同时使用内存。假设有两个进程，分别是进程 A 和进程 B，通过分时复用的方式运行在一个处理器上，两个进程都需要使用内存。

第一种简单的方法是：当进程 A 运行时，操作系统允许它访问所有的物理内存资源；当发生进程切换时，操作系统将进程 A 的所有内存数据保存到存储设备（如磁盘）中，然后将进程 B 的数据从存储设备加载到内存中，再运行进程 B。使用这种方法，两个进程都可以使用所有的物理内存，能保证相互隔离，没有冲突。但同样存在明显的弊端，即读写存储设备的速度很慢，这将导致切换程序的时间开销过大。

第二种简单的方法是：让每个进程独立使用物理内存的一部分，地址范围互不覆盖；数据一直驻留在内存中，这样在进程切换时不再需要访问存储设备。该方法在性能方面优于前一种方法，但是也存在两个严重的弊端：第一，无法保证不同进程使用的物理内存之间的隔离性，比如进程 A 在运行过程中可能意外地写了进程 B 的物理内存，导致后者运行错误，或是恶意窃取进程 B 的机密数据；第二，无法保证进程可用地址空间的连续性和统一性，程序在不同机器甚至同一台机器上的多次运行所使用的地址空间可能都不一样，这将大大增加程序编写及编译的复杂性。

第三种方法是在应用程序与物理内存之间加入一个新的抽象——虚拟内存，使应用程序能够方便地、安全地使用物理内存资源。引入虚拟内存后，应用程序不再直接访问内存的物理地址，而是通过虚拟地址来间接访问物理内存。这些虚拟地址会被处理器翻译为物理地址，然后再访问对应的物理内存。程序员只需要面向虚拟地址来编写应用程序，应用程序在运行时也只能使用虚拟地址。从虚拟地址到物理地址的地址翻译是由处理器与操作系统协同完成的：处理器负责在程序运行时将虚拟地址动态翻译为物理地址，操作系统则负责配置虚拟地址与物理地址之间的具体映射。如图 3.16 所示，进程所观察到的是一段连续的虚拟地址空间。一块连续虚拟地址的区域可以映射到多块不连续的物理内存区域，对于没有使用的虚拟地址区域，操作系统可以不做映射，从而节省物理内存。

与进程类似，我们也可以直接通过简单的代码来验证不同进程的虚拟内存地址空间是独立的。我们编写一段打印全局变量的值与地址的程序，并同时创建两个进程，如代码片段 3.8 所示。

代码片段 3.8 中的两个应用程序都是循环打印一个全局变量的地址和值。在编译时利用 `--static` 参数，可保证每次执行程序时全局变量的位置都是固定的。通过同时启动两个进程来运行这一程序可以发现，打印的内存地址一样，但是内存中的值不一样（见输出记录 3.3）。由此可见，两个进程的地址空间是彼此独立的。

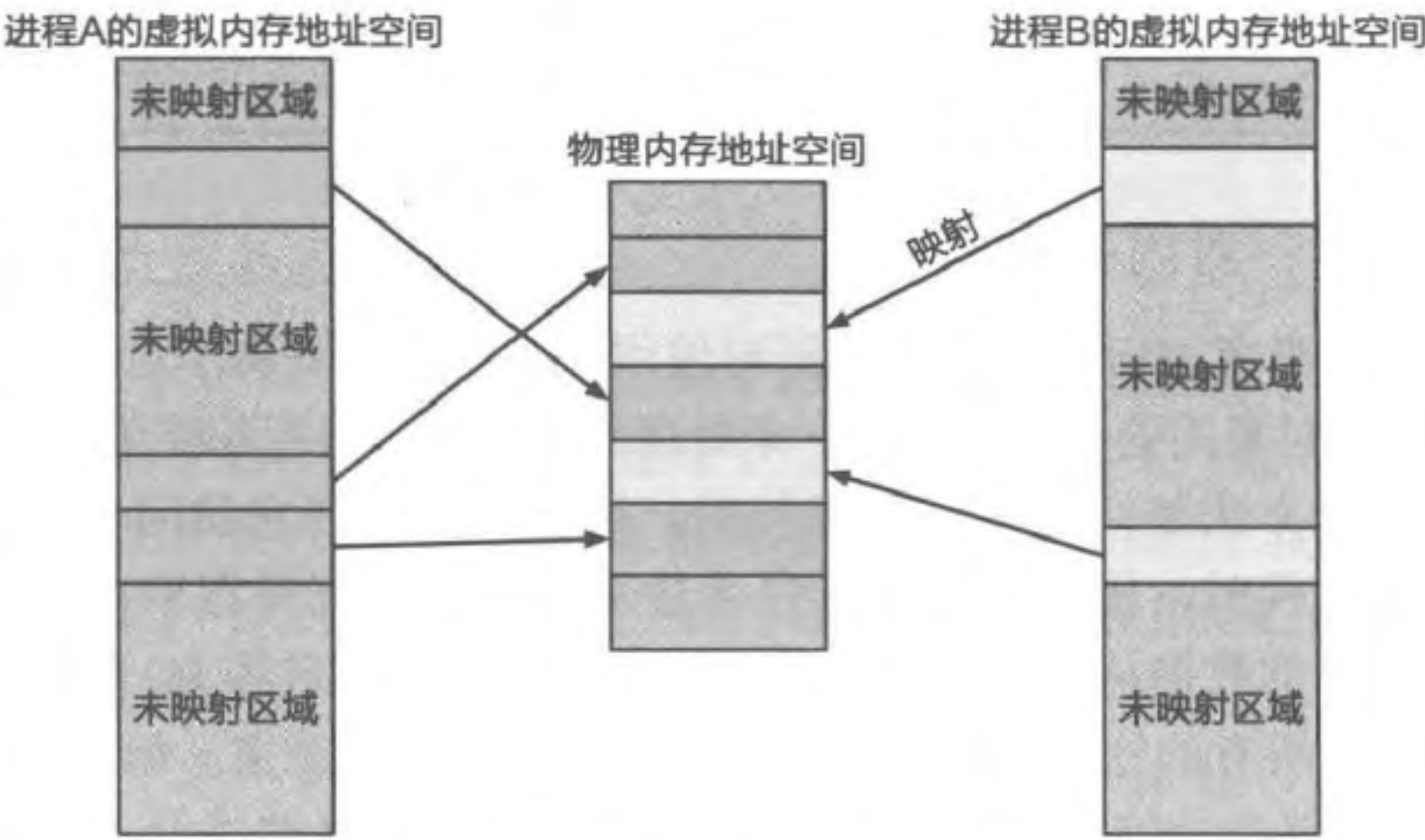


图 3.16 程序观察到的虚拟内存空间并非实际上的物理内存

代码片段 3.8 不同应用程序的地址空间彼此独立

```
1 // file: vm-example.c
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5
6 int g_val = 0;
7
8 int main(int argc, char** argv)
9 {
10     g_val = strtol(argv[1], NULL, 10);
11     pid_t pid = getpid();
12     for (int i = 0; i < 2; i++) {
13         printf("[%d]The address of g_val is %p.\n", pid, &g_val);
14         printf("[%d]The value of g_val is %d.\n", pid, g_val);
15         sleep(1);
16     }
17 }
```

输出记录 3.3 同时启动多个 vm-example 的输出

```
[user@osbook ~] $ gcc vm-example.c --static -o vm-example
[user@osbook ~] $ ./vm-example 1 & ./vm-example 2 & wait
[5021]The address of g_val is 0x4a7330.
[5021]The value of g_val is 1.
[5022]The address of g_val is 0x4a7330.
[5022]The value of g_val is 2.
[5021]The address of g_val is 0x4a7330.
[5021]The value of g_val is 1.
[5022]The address of g_val is 0x4a7330.
[5022]The value of g_val is 2.
```

虚拟内存这一抽象为应用程序带来了诸多好处。

- 程序员在编程时，无须关心物理内存具体如何分配，也不用关心所使用的地址是否会与其他进程重叠，而是可以使用连续的整个虚拟地址空间^①。实际运行时的内存资源分配由操作系统负责，进而大大减轻了程序员的负担。
- 每个应用程序的虚拟地址空间是彼此隔离的，因此应用程序在运行期间的内存读写对其他应用程序是不可见的，这样就保证了不同应用程序之间的隔离，防止出错程序或恶意程序干扰或破坏其他程序的执行。
- 操作系统可以选择只将程序实际正在使用的虚拟地址映射到物理内存地址，对于从未被进程使用的虚拟地址，操作系统还可以不将其映射到任何位置，从而提高内存的资源利用率。
- 操作系统可以选择将部分虚拟内存区域的数据暂存到磁盘上，从而允许应用程序使用的内存大小突破物理内存的容量限制。
- 操作系统可以为不同的虚拟内存区域设置不同的权限，包括可读、可写、可执行，以及是允许用户态访问还是仅内核态可访问等，从而增强程序执行和系统整体的安全性。除此之外，操作系统还可以通过虚拟内存实现一些更加高级的功能，例如写时拷贝、程序间的内存共享等，本书将在后续章节中进一步展开讨论。

3.3.4 进程的虚拟内存布局

每个进程都有自己独立的虚拟内存空间，其布局大体都是相同的。图 3.17 展示了一种典型的虚拟内存空间布局，其中，每个进程的虚拟内存空间都是一段从 0 开始的连续地址空间。从低地址到高地址的分布如下：

- 代码段与数据段。数据段与代码段通常位于虚拟地址空间中较低的地址。其中，数据段主要保存的是全局变量的值，代码段保存的则是执行代码。这两个部分都保存在可执行文件中，在进程执行前，操作系统会将它们载入虚拟地址空间。
- 用户堆。堆管理的是进程在运行过程中动

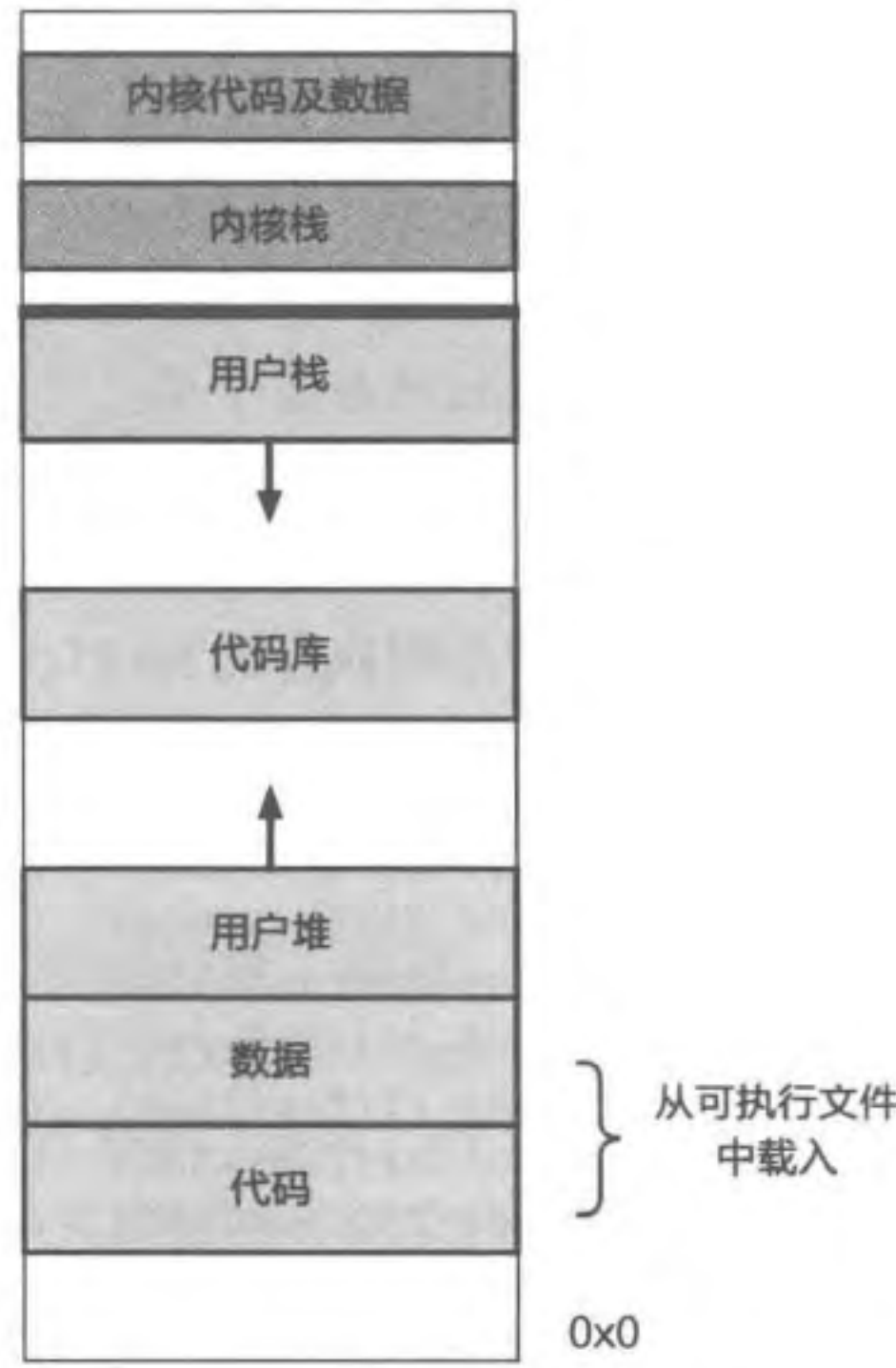


图 3.17 典型的虚拟内存空间布局

① 严格来说不是所有虚拟地址都能使用，例如零地址通常无法使用。若内核与应用共享地址空间，则内核占据的地址段也无法使用。

态分配的内存，例如通过 `malloc` 分配的区域。堆的扩展方向是自底向上：堆顶在高地址，当进程需要更多内存时，堆顶会向高地址扩展。

- 代码库。进程的执行有时需要依赖共享的代码库（比如 `libc`），这些代码库会被映射到用户栈下方的虚拟地址，并标记为只读。
- 用户栈。栈保存了进程需要使用的各种临时数据（如临时变量的值）。栈是一个可以伸缩的数据结构，其扩展方向是自顶向下：栈底在高地址，栈顶在低地址。当临时数据被压入栈内时，栈顶会向低地址扩展。
- 内核部分。进程地址空间顶端的部分通常是操作系统内核保留的区域，又可以进一步细分为内核代码段、内核数据段、内核栈等。应用程序无法直接访问这部分内存区域，只有操作系统内核才能访问。对于不同进程来说，这部分虚拟地址空间的映射都是一样的。

小知识

将操作系统内核的虚拟地址区域映射到所有进程的虚拟地址空间只是一种设计上的选择，而不是必需的。这么做的好处在于当进程发起一个系统调用下陷到操作系统时，不需要切换虚拟地址空间，性能更好一些；此外，操作系统在处理系统调用时，也可以方便地访问用户态的数据。缺点在于应用程序与内核的隔离性不够强，例如内核可能不小心跳转到用户态的代码段执行，或者将一些关键数据误写入用户态的数据段。另一种设计是为操作系统内核单独创建一个独立的虚拟地址空间，可以提高隔离能力，但性能相对会差一些。

在 Linux 中，用户可以通过 `cat /proc/PID/maps` 来查看某个进程的内存空间布局。比如，对于用户小明执行的 `hello`，其进程对应的 `maps` 内容可能如输出记录 3.4 所示。

输出记录 3.4 Linux 中内存空间布局的例子

ffffc2122000-ffffc2143000	[stack]
ffffae6a8000-ffffae6ab000	[/lib/aarch64-linux-gnu/ld-2.24.so]
ffffae6a7000-ffffae6a8000	[vdso]
ffffae6a6000-ffffae6a7000	[vvar]
ffffae699000-ffffae69b000	(anonymous)
ffffae67d000-ffffae699000	[/lib/aarch64-linux-gnu/ld-2.24.so]
ffffae679000-ffffae67d000	(anonymous)
ffffae533000-ffffae679000	[/lib/aarch64-linux-gnu/libc-2.24.so]
aaaadb9ec000-aaaadba0d000	[heap]
aaaab2607000-aaaab2609000	[/home/xiaoming/hello] (data)
aaaab25f7000-aaaab25f8000	[/home/xiaoming/hello] (code)

在上面的例子中，由于内核地址空间对用户进程不可见，因此 `maps` 内容没有包含内核部分的映射。除此之外，`maps` 内容与图 3.17 基本吻合，但还有一些额外的区域，例如

vdso 和 vvar 是与系统调用相关的内存区域。另外，进程也会映射一些匿名内存区域用于完成缓存、共享内存等工作，这里不展开介绍。

3.3.5 文件：对存储设备的抽象

对于大部分应用程序来说，仅仅有处理器和内存是远远不够的，因为内存中的数据在计算机重启后就完全消失了。对用户而言，他们也有许多数据想要永久存储在计算机系统中，如日常生活中的文档、视频、程序等。现代计算机系统有着非常多样的存储设备，例如桌面主机中常见的机械硬盘与固态硬盘，以及移动端的嵌入式多媒体卡和通用闪存存储。

为了将不同存储设备的细节隐藏起来，让程序员可以通过统一、便利的方式来访问这些设备，操作系统使用了文件这一抽象。文件是一个“有名字的字节序列”，每个文件都拥有自己的文件名，文件名独立于文件的数据而存在。不同的文件名可以属于同一个目录，目录可以层层嵌套，顶层是根目录，通常用“/”表示。

图 3.18 展示了 Linux 系统典型的目录结构。以其中的 hello 程序为例，当在 shell 中执行 ./hello 程序时，操作系统会根据字符串“hello”，在 /home/user 目录下找到对应的文件名，并进一步找到 hello 文件在存储设备上的位置，最后将其加载到内存中并执行。

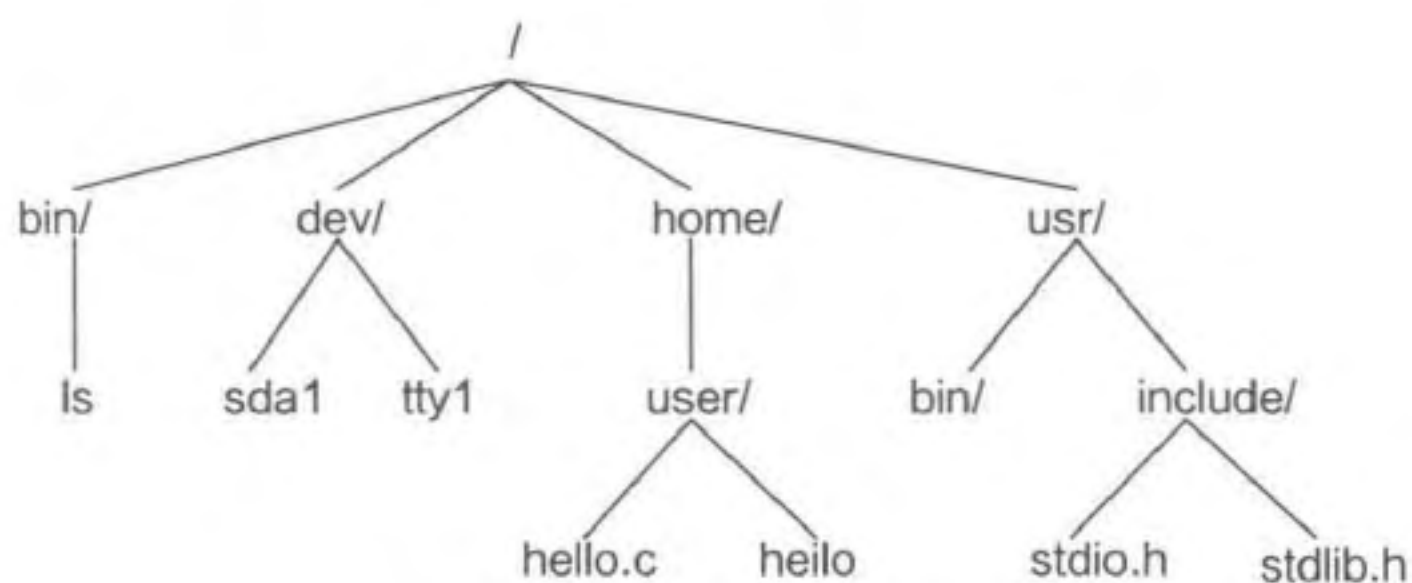


图 3.18 Linux 层级结构的例子

操作系统提供了一组接口用于文件访问，包括文件的打开、读取、写入、关闭等。例如，应用程序可以通过 open 接口打开一个文件。在打开成功后，open 会返回一个文件描述符（File Descriptor, fd）。通过文件描述符，可以利用 read、write 接口分别对相应的文件进行读取和写入操作，并最终通过 close 关闭已打开的文件。代码片段 3.9 展示了使用这些接口来读取文件，并用 write 代替 printf 输出到屏幕的例子。

代码片段 3.9 利用文件系统接口来读取文件的例子

```

1 #include <fcntl.h>
2 #include <stdio.h>
3 #include <unistd.h>
4

```



```
5 int main()
6 {
7     // 打开文件 hello.txt, O_RDONLY 表示以只读方式打开
8     int fd = open("hello.txt", O_RDONLY);
9
10    char result[14];
11    // 从 fd 对应的文件中读取 13 字节
12    read(fd, result, 13);
13    // 向屏幕输出读取的字符串
14    write(1, result, 13);
15    close(fd);
16 }
```

除了通过 read 与 write 接口来访问文件外，操作系统也支持直接将文件映射到虚拟地址中来访问——这种方式称为内存映射，具体为 mmap 系统调用。通过将文件与一块连续的虚拟内存区域相关联，用户程序可以直接通过读写内存的方式来访问文件。

图 3.19 展示了使用 mmap 来映射一个文本文件的例子。如代码片段 3.10 所示，首先通过 open 接口打开 hello.txt 文件，然后通过 mmap 将这一文件中的 13 个字符映射到内存中，并将得到的内存映射区域存放在 start 指针中。PROT_READ 参数表明这段内存映射区域是只读的，同时 MAP_PRIVATE 参数表明这段内存映射区域是进程私有的，不能被其他进程操作[Ⓐ]。我们将 start 指向的内容输出，便能发现其与文件内容是一致的，如输出记录 3.5 所示。

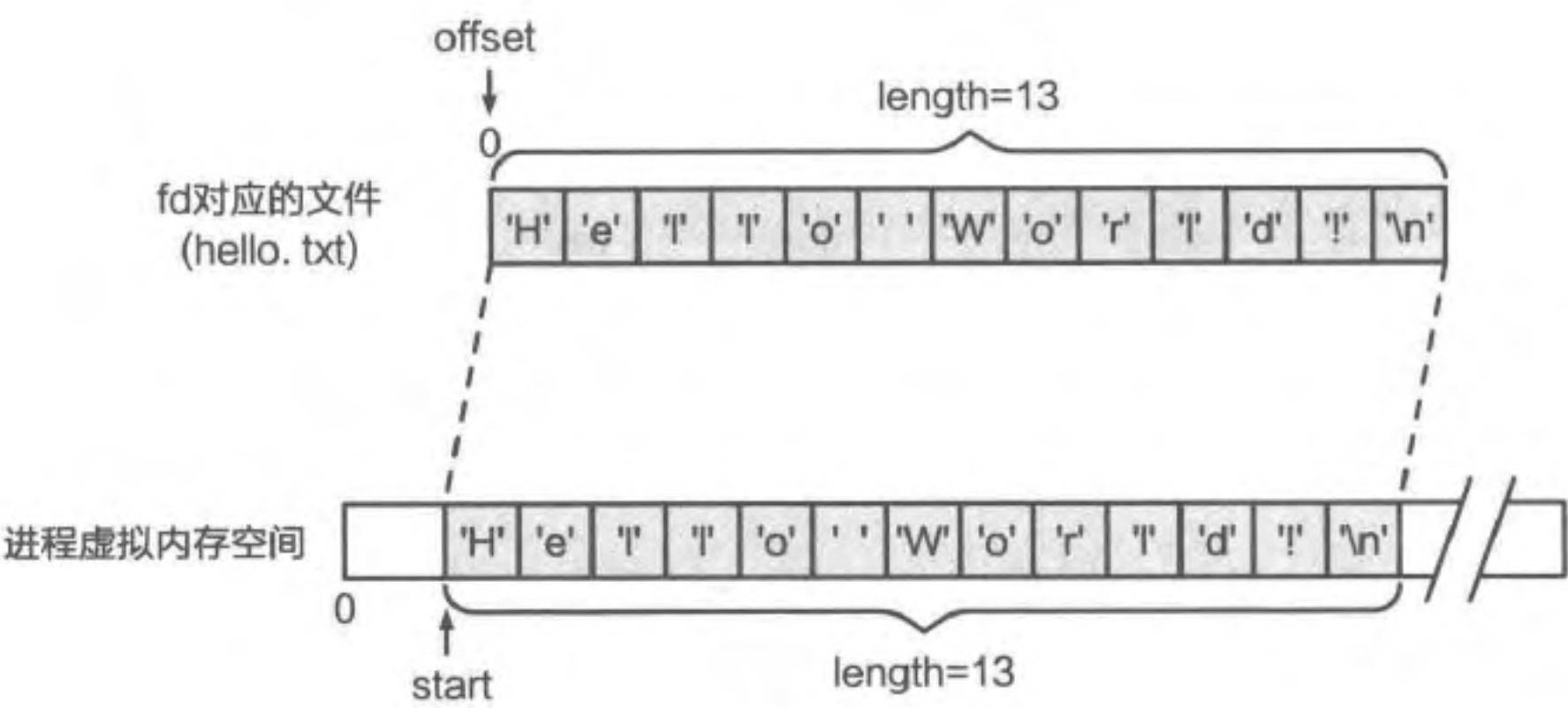


图 3.19 使用 mmap 映射文本文件的例子

代码片段 3.10 利用内存映射来读取文件

```
1 #include <fcntl.h>
2 #include <sys/mman.h>
```

Ⓐ 除了 MAP_PRIVATE 外，mmap 还可以指定其他不同的内存映射类型，从而实现进程间内存共享、匿名内存等许多有趣的功能，我们将在后续章节中进行介绍。

```

3 #include <unistd.h>
4
5 int main()
6 {
7     int fd = open("hello.txt", O_RDONLY);
8     void *start = mmap(NULL, 13, PROT_READ, MAP_PRIVATE, fd, 0 /* offset */);
9     write(1, start, 13);
10 }

```

输出记录 3.5 代码片段 3.10 的输出

```

[user@osbook ~] $ ./hello-mmap
Hello World!
[user@osbook ~] $

```

3.3.6 文件：对所有设备的抽象

对 I/O 设备的抽象是操作系统面临的一个巨大挑战。不同于处理器和内存，I/O 设备的特点是种类多、差异大，是否存在一种“one size fits all”的统一抽象，可以为所有的设备提供同一组通用的接口呢？

以 Linux 为代表的操作系统继承了 UNIX 经典的设计哲学——“一切都是文件”（everything is a file），将文件作为所有 I/O 设备的抽象。这里主要有两方面的原因：一方面，从用户的角度来看，许多设备确实有与文件类似的操作，比如对串口设备的读写操作，可以很自然地对应到对文件的读写操作；另一方面，从实现的角度来看，将设备抽象为文件可以复用操作系统中对文件进行操作的大量代码。

在 Linux 中，为了让应用程序能够访问某个设备，会首先为这个设备创建一个特殊的文件——设备文件，通常放在 `/dev/` 目录下。如果应用程序需要操作这个设备，则可以通过 `open` 打开这个设备文件，获得对应的 `fd`，并进一步进行读写操作。例如，程序员平时常用的终端，实际上便是在与一类叫作 TTY 设备^①的特殊文件打交道。在 `hello` 程序中使用 `printf` 向终端进行输出时，实际上是通过 `write` 系统调用将需要输出的内容写入 TTY 设备文件中。类似地，`mysh` 在通过 `fgets` 读取用户输入时，实际上也是通过 `read` 系统调用从 TTY 设备文件中读取键盘的输入。这一特殊文件的打开与关闭，是在进程创建与退出时由操作系统自动完成的。

我们可以编写程序，直接通过 `write` 系统调用来向当前终端甚至别的终端直接输出内容。在图 3.20 所示的例子中，我们启动了两个终端。在第一个终端中，通过 `tty` 命令可以知道这一终端对应的 TTY 设备文件。随后，我们编写并编译代码片段 3.11，并在第二个终端中执行。这样在第一个终端中，便能在没有输入任何命令的情况下，“凭空”输

^① TTY 是 Teletypes 或 Teletypewriter 的缩写。

出我们刚刚写入的字符。



图 3.20 两个终端的输出示例

代码片段 3.11 通过 write 直接向 TTY 设备文件写入内容，实现跨终端输出

```
1 // file: hello-tty.c
2 #include <unistd.h>
3 #include <sys/fcntl.h>
4 int main()
5 {
6     // 打开另一个终端的 TTY 设备文件，O_WRONLY 表示允许写入
7     int fd = open("/dev/pts/18", O_WRONLY);
8     // 向该文件进行输出
9     write(fd, "Hello World!\n", 13);
10    // 关闭文件
11    close(fd);
12 }
```

然而，并非所有设备都适合被抽象为文件并通过 read、write 接口来操作。例如，如果将音频数据写入声音设备就能播放该音频的话，那么调整音量应该如何操作呢？如果将图像数据写入显卡就能显示图像，那么调节分辨率、刷新率该如何操作呢？为了实现这些并不标准的操作，操作系统提供了更加通用的底层接口 ioctl，允许应用程序以更灵活的方式向设备发送命令与数据。对于一些常用的设备文件类型，如网卡设备，操作系统还专门提供了套接字（Socket）的抽象与相应的接口，使应用能够更方便地使用这些设备。我们将在第 11 章与第 13 章中详细介绍文件系统与设备。

3.4 思考题

1. 为什么右移要分算术右移和逻辑右移，而左移只有逻辑左移？
2. 本章只介绍了通过对控制流的条件控制实现条件执行的方式，但部分其他 ISA 同样支持对数据流的条件控制。具体来讲，这种方式使得数据搬移指令像分支指令一样实现条件执行，以此控制数据流的流向。请分析这种条件执行方式与控制流条件控制方式的优劣。
3. 分类讨论哪些局部变量会被存储到运行时栈上的局部变量区。